

I. PENDAHULUAN

A. Deskripsi Modul

Modul ini membahas fondasi utama dalam pengambilan data (Data Retrieval) menggunakan SQL Server. Berbeda dengan pendekatan sintaksis biasa, modul ini menekankan pada Logical Query Processing, yakni bagaimana mesin database sebenarnya memproses perintah SQL langkah demi langkah. Pemahaman ini krusial untuk menulis query yang efisien dan akurat..

B. Tujuan Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

1. Membedakan dengan jelas antara DML (Data Manipulation) dan DDL (Data Definition).
2. Memahami perbedaan antara urutan penulisan (Lexical Order) dan urutan eksekusi (Logical Order) query.
3. Menggunakan statement SELECT, FROM, WHERE, GROUP BY, HAVING, dan ORDER BY dengan benar.
4. Menerapkan fungsi agregasi, filterisasi, DISTINCT, TOP, dan OFFSET FETCH.
5. Menganalisis hasil query berdasarkan studi kasus data retail.

C. Prasyarat

1. Mahasiswa telah menyelesaikan modul DDL (memahami CREATE TABLE dan CONSTRAINT).
2. Microsoft SQL Server Management Studio (SSMS)/ DataGrip telah terinstal dan mahasiswa memiliki akses login ke instance SQL Server.
3. Pemahaman dasar tentang tabel, baris, dan kolom..

II. LANDASAN TEORI: LOGICAL QUERY PROCESSING

Dalam SQL Server, terdapat perbedaan mendasar antara urutan **penulisan kode** (*Lexical Order*) dengan urutan **eksekusi mesin** (*Logical Order*). Memahami diagram *Logical Query Processing* adalah kunci untuk melakukan *debugging* dan optimasi query.

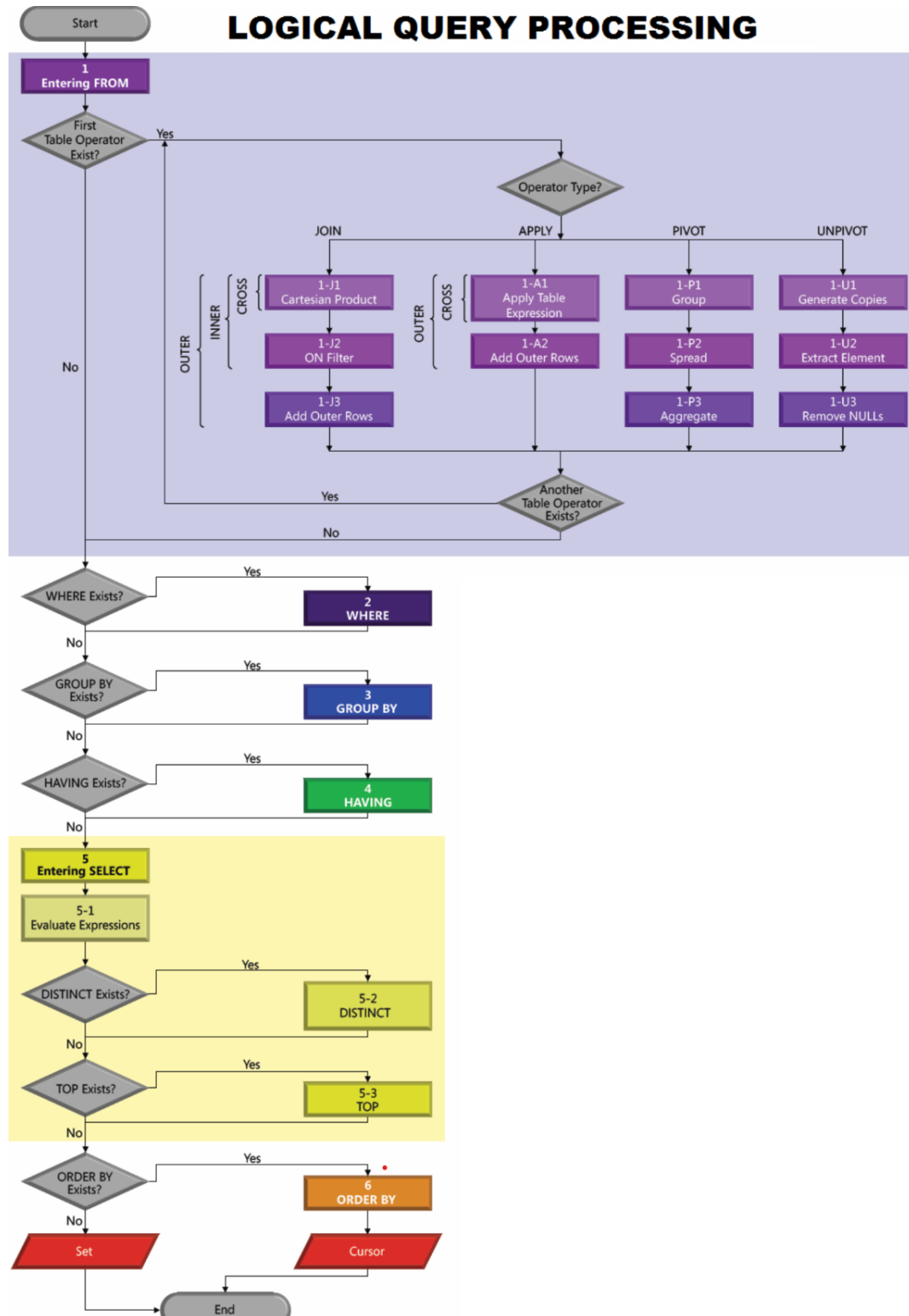
Konsep "Virtual Table"

Setiap langkah dalam pemrosesan query (nomor 1 s.d. 6 pada gambar 1) menerima masukan berupa tabel (data mentah) dan menghasilkan keluaran berupa **Virtual Table (VT)**. VT ini bersifat sementara di memori dan menjadi masukan untuk langkah berikutnya. Pengguna (user) hanya melihat hasil akhir, namun mesin memproses VT ini secara bertahap.

Perbandingan Urutan (Lexical vs Logical)

Urutan Penulisan (Lexical)	Urutan Eksekusi (Logical)	Deskripsi Singkat
1. SELECT	5. SELECT	Memilih kolom yang akan ditampilkan.
2. FROM	1. FROM	Menentukan sumber data/tabel.
3. WHERE	2. WHERE	Memfilter baris (Row Filter).

Urutan Penulisan (Lexical)	Urutan Eksekusi (Logical)	Deskripsi Singkat
4. GROUP BY	3. GROUP BY	Mengelompokkan data.
5. HAVING	4. HAVING	Memfilter grup (Group Filter).
6. ORDER BY	6. ORDER BY	Mengurutkan hasil akhir.



Gambar 1 Logical Query Process

PEMBEDAHAN SETIAP LANGKAH (STEP-BY-STEP)

LANGKAH 1: FROM (Entering FROM)

Mengacu pada diagram: Bagian ungu atas (Cartesian Product, Join, Apply, Pivot).

Ini adalah fase inisialisasi. SQL Server harus tahu "Dari mana data ini berasal?".

- **Proses:** Mesin memvalidasi keberadaan tabel di database.
- **Pada Diagram:** Jika ada operasi tabel (seperti JOIN), proses Cartesian Product (1-J1), Filter ON (1-J2), dan penambahan Outer Rows (1-J3) terjadi di sini. Namun, untuk *Single Table* (satu tabel), prosesnya sederhana: mesin memuat seluruh tabel sumber ke dalam memori kerja sebagai **Virtual Table 1 (VT1)**.
- **Poin Kritis:** Karena SELECT belum dijalankan, mesin **belum tahu** nama-nama kolom alias yang Anda buat. Inilah sebabnya Anda tidak bisa menggunakan alias kolom di WHERE clause.

LANGKAH 2: WHERE (Row Filter)

Mengacu pada diagram: Diamond "WHERE Exists?"

Fase ini adalah "Gerbang Pertama". Tujuannya adalah mengurangi jumlah data sesegera mungkin untuk efisiensi.

- **Input:** VT1 dari langkah FROM.
- **Logika:** Predikat Logika (Kondisi).
 - Setiap baris dievaluasi: Apakah kondisinya TRUE?
 - Jika TRUE: Baris masuk ke **VT2**.
 - Jika FALSE atau UNKNOWN (NULL): Baris dibuang.
- **Implikasi:** Di sini kita memfilter data individu. Misalnya WHERE Salary > 5000. Data yang belum dikelompokkan disaring di sini.

LANGKAH 3: GROUP BY (Grouping)

Mengacu pada diagram: Diamond "GROUP BY Exists?"

Fase ini mengubah bentuk data dari "Detail" menjadi "Ringkasan" (Aggregate).

- **Input:** VT2 dari langkah WHERE.
- **Proses:** Baris-baris dengan nilai yang sama pada kolom yang ditentukan akan disatukan menjadi satu baris representatif (satu grup).
- **Output:** VT3.
- **Aturan Emas (The Golden Rule):** Setelah langkah ini, elemen yang boleh ada di langkah selanjutnya (HAVING, SELECT) hanyalah:
 1. Kolom yang menjadi pengelompok (tercatat di GROUP BY).
 2. Fungsi Agregat (SUM, COUNT, AVG, MIN, MAX).

Anda tidak bisa lagi memanggil kolom detail (misal: NamaPegawai) jika tidak masuk dalam grouping.

LANGKAH 4: HAVING (Group Filter)

Mengacu pada diagram: Diamond "HAVING Exists?"

Fase ini adalah "Gerbang Kedua". HAVING identik dengan WHERE, bedanya adalah waktu kejadiannya.

- **Input:** VT3 (Data yang sudah berupa grup).
- **Logika:** Mengevaluasi predikat pada level grup.
- **Contoh:** HAVING COUNT(*) > 5. Mesin melihat hasil agregasi VT3, lalu membuang grup yang jumlah anggotanya kurang dari atau sama dengan 5.
- **Output:** VT4.

LANGKAH 5: SELECT (Processing Select List)

Mengacu pada diagram: Kotak kuning besar (5 Entering SELECT).

Baru di tahap inilah kita menentukan "Bentuk Akhir" tabel yang ingin dilihat user. Langkah ini memiliki sub-fase penting sesuai diagram:

- **5-1 Evaluate Expressions:** Mesin menghitung ekspresi aritmatika (misal: Price * Qty) atau manipulasi string. Alias kolom (AS NamaBaru) didefinisikan di sini.
- **5-2 DISTINCT:** *Jika ada.* Mesin menghapus baris duplikat dari hasil SELECT. Ini operasi yang memakan *resource* (mahal) karena seringkali membutuhkan pengurutan internal sementara.
- **5-3 TOP:** *Jika ada.* Mesin memotong hasil berdasarkan jumlah atau persentase yang diminta.
- **Output:** VT5.

LANGKAH 6: ORDER BY (Presentation Sorting)

Mengacu pada diagram: Kotak oranye (ORDER BY) dan jajar genjang Cursor.

Ini adalah satu-satunya langkah di mana hasil query dijamin urutannya.

- **Input:** VT5.
- **Proses:** Mengurutkan data berdasarkan kolom yang diminta (ASC/DESC).
- **Keunikan:** Karena Langkah 6 terjadi **setelah** Langkah 5 (SELECT), maka **Anda BOLEH menggunakan Alias Kolom di ORDER BY.**
 - *Contoh:* SELECT Salary AS Gaji ... ORDER BY Gaji DESC (Valid).
- **Output Akhir:** Sebuah **Cursor** (objek yang bisa dinavigasi klien) dikirimkan ke aplikasi pengguna (SSMS/Web App).

Catatan penting

Cara berpikir Anda dalam membuat query:

"Pertama, saya ambil tabel apa? (**FROM**) -> Lalu baris mana yang saya buang? (**WHERE**) -> Apakah saya perlu meringkasnya jadi grup? (**GROUP BY**) -> Grup mana yang lolos seleksi? (**HAVING**) -> Kolom mana yang mau saya tampilkan dan hitung? (**SELECT**) -> Terakhir, bagaimana urutannya? (**ORDER BY**)."

III. PERSIAPAN LABORATORIUM

Pada sesi ini, kita akan menggunakan tabel:

1. **Sales.SalesOrderDetail**: Tabel transaksi detail (item apa dibeli, jumlah, harga).
2. **Production.Product**: Tabel master barang.

BAGIAN 1: SELECT, FROM, & DASAR KOLOM (Step 1 & 5)

Perintah dasar untuk mengambil data. Ingat, secara logika FROM dieksekusi sebelum SELECT.

Sintaks Dasar:

SQL

```
SELECT [DISTINCT] column1, column2
FROM table_name;
```

Latihan dan Analisa

Latihan 1.1: Menampilkan Seluruh Data Produk Ambil semua kolom dari tabel produk.

SQL

```
SELECT * FROM Production.Product;
```

Analisa: Simbol * meminta SQL Server mengambil semua kolom. Gunakan ini hanya untuk eksplorasi, hindari di aplikasi produksi karena boros *resource*.

Latihan 1.2: Menampilkan Kolom Spesifik Tampilkan hanya nama produk, nomor produk, dan harga jual (ListPrice).

SQL

```
SELECT Name, ProductNumber, ListPrice
FROM Production.Product;
```

Analisa: Ini disebut *Projection*. Kita membatasi kolom yang diambil pada Step 5 (SELECT) setelah Step 1 (FROM) memuat tabel.

Latihan 1.3: Menggunakan Alias Kolom Tampilkan nama produk dan harga jual dengan nama kolom 'Nama Barang' dan 'Harga Jual'.

SQL

```
SELECT Name AS [Nama Barang], ListPrice AS 'Harga Jual'
FROM Production.Product;
```

Analisa: Alias (AS) mengubah nama header hasil output agar lebih mudah dibaca manusia.

Latihan 1.4: Kalkulasi Sederhana (Derived Column) Tampilkan nama produk dan estimasi harga jika dinaikkan 10%.

SQL

```
SELECT Name, ListPrice, (ListPrice * 1.1) AS HargaBaru
FROM Production.Product;
```

Analisa: Ekspresi matematika dievaluasi pada Step 5-1. Data asli di tabel tidak berubah, hanya tampilan hasil yang berubah.

Latihan 1.5: Menggabungkan String Tampilkan nama produk digabung dengan nomor produk dalam satu kolom.

SQL

```
SELECT Name + ' (' + ProductNumber + ')' AS ProdukLengkap
FROM Production.Product;
```

Analisa: Operator + pada SQL Server digunakan untuk konkatenasi string.

BAGIAN 2: FILTERISASI DATA DENGAN WHERE (Step 2)

WHERE memfilter baris **sebelum** pengelompokan atau pemilihan kolom. Ini sangat efisien untuk mengurangi beban kerja server. Adapun operator yang dapat digunakan dalam memfilter, yakni:

1. **Operator Pemanding:** =, <>, >, <, >=, <=
2. **Operator Logika:** AND (kedua syarat harus benar), OR (salah satu syarat benar).
3. **Pencarian Pola:** LIKE (menggunakan % sebagai wildcard).
4. **List:** IN (mencocokkan nilai dalam daftar).

Latihan dan Analisa

Latihan 2.1: Filter Kondisi Tunggal (Exact Match) Cari produk yang warnanya 'Red'.

SQL

```
SELECT Name, Color, ListPrice
FROM Production.Product
WHERE Color = 'Red';
```

Analisa: SQL mengecek setiap baris dari Step 1 (FROM). Jika kolom Color berisi 'Red', baris diteruskan ke langkah berikutnya.

Latihan 2.2: Filter Numerik (Range) Cari produk dengan harga jual di atas 1000.

SQL

```
SELECT Name, ListPrice
FROM Production.Product
WHERE ListPrice > 1000;
```


Analisa: Menggunakan operator perbandingan matematika.

Latihan 2.3: Kondisi Jamak (AND) Cari produk berwarna 'Black' DAN harganya di atas 500.

SQL

```
SELECT Name, Color, ListPrice
FROM Production.Product
WHERE Color = 'Black' AND ListPrice > 500;
```

Analisa: Operator AND mengharuskan **kedua** kondisi bernilai TRUE agar baris ditampilkan.

Latihan 2.4: Kondisi Jamak (OR & IN) Cari produk yang warnanya 'Red', 'Blue', atau 'Black'.

SQL

```
SELECT Name, Color
FROM Production.Product
WHERE Color IN ('Red', 'Blue', 'Black');
-- Cara lain: WHERE Color = 'Red' OR Color = 'Blue' OR Color = 'Black'
```

Analisa: IN adalah bentuk singkat dari beberapa OR. Baris akan diambil jika warnanya cocok dengan salah satu dari daftar.

Latihan 2.5: Filter Pola String (LIKE) Cari produk yang namanya mengandung kata 'Road'.

SQL

```
SELECT Name, ProductNumber
FROM Production.Product
WHERE Name LIKE '%Road%';
```

Analisa: % adalah *wildcard*. %Road% berarti "apapun di depan, ada kata Road, dan apapun di belakang".

BAGIAN 3: AGREGASI & PENGELOMPOKAN (GROUP BY) (Step 3)

GROUP BY memampatkan banyak baris menjadi satu baris ringkasan berdasarkan kolom tertentu.

Latihan dan Analisa

Latihan 3.1: Menghitung Total Baris (COUNT) Hitung berapa banyak total produk yang terdaftar.

SQL

```
SELECT COUNT(*) AS TotalProduk
FROM Production.Product;
```

Analisa: Agregat skalar tanpa GROUP BY. Menghitung semua baris yang lolos dari FROM.

Latihan 3.2: Pengelompokan Sederhana Tampilkan daftar warna yang tersedia dan berapa jumlah produk untuk setiap warna.

SQL

```
SELECT Color, COUNT(*) AS JumlahProduk
FROM Production.Product
GROUP BY Color;
```

Analisa: Pada Step 3, SQL mengumpulkan baris dengan warna sama, lalu fungsi COUNT menghitung jumlah anggota di setiap kelompok warna tersebut.

Catatan: Perhatikan hasil NULL (produk tanpa warna). NULL dianggap sebagai satu grup tersendiri.

Latihan 3.3: Agregasi Numerik (SUM, AVG) Gunakan tabel Sales.SalesOrderDetail. Hitung total kuantitas barang (OrderQty) dan rata-rata harga satuan (UnitPrice) per ID Produk.

SQL

```
SELECT ProductID, SUM(OrderQty) AS TotalTerjual, AVG(UnitPrice) AS
RataRataHarga
FROM Sales.SalesOrderDetail
GROUP BY ProductID;
```

Analisa: Kita melihat performa penjualan per produk. SUM menjumlahkan, AVG mencari rata-rata.

Latihan 3.4: Grouping Lebih dari Satu Kolom Di tabel Production.Product, kelompokkan berdasarkan Warna (Color) dan Ukuran (Size).

SQL

```
SELECT Color, Size, COUNT(*) AS Jumlah
FROM Production.Product
GROUP BY Color, Size;
```

Analisa: Grup dibuat berdasarkan kombinasi unik Warna dan Ukuran (misal: Red-L, Red-M, Black-L, dst).

Latihan 3.5: Validasi Logika Grouping Cobalah query berikut (akan error) dan analisa penyebabnya:

SQL

```
-- QUERY ERROR
SELECT Color, Name, COUNT(*)
FROM Production.Product
GROUP BY Color;
```

Analisa: Error. Karena dikelompokkan berdasarkan Color, kolom Name tidak bisa ditampilkan secara mentah karena satu warna (misal Red) memiliki banyak Nama Produk. Kolom di SELECT haruslah kolom yang ada di GROUP BY atau berada dalam fungsi agregat.

BAGIAN 4: FILTER HASIL AGREGASI (HAVING) (Step 4)

HAVING adalah "WHERE" untuk data yang sudah di-group. Ingat diagram: WHERE (Step 2) -> GROUP BY (Step 3) -> HAVING (Step 4).

Latihan dan Analisa

Latihan 4.1: Filter Group Dasar Tampilkan warna produk yang memiliki jumlah produk lebih dari 20 item.

SQL

```
SELECT Color, COUNT(*) AS Jumlah
FROM Production.Product
GROUP BY Color
HAVING COUNT(*) > 20;
```

Analisa: SQL menghitung dulu jumlah produk per warna. Setelah angkanya ketemu, baru difilter mana yang > 20. Kita tidak bisa menggunakan WHERE COUNT(*) > 20 karena WHERE terjadi sebelum penghitungan.

Latihan 4.2: Kombinasi WHERE dan HAVING Tampilkan Warna yang jumlah produknya > 10, TAPI hanya untuk produk yang harganya > 500.

SQL

```
SELECT Color, COUNT(*) AS Jumlah
FROM Production.Product
WHERE ListPrice > 500 -- Filter baris dulu (Step 2)
GROUP BY Color -- Kelompokkan sisa baris (Step 3)
HAVING COUNT(*) > 10; -- Filter hasil kelompok (Step 4)
```

Analisa:

1. Buang produk murah (≤ 500).
2. Kelompokkan sisanya berdasarkan warna.
3. Tampilkan warna yang anggotanya masih > 10.

Latihan 4.3: Filter Berdasarkan Total Penjualan Dari Sales.SalesOrderDetail, cari ProductID yang total kuantitas penjualannya (OrderQty) di atas 1000 unit.

SQL

```
SELECT ProductID, SUM(OrderQty) AS TotalQty
FROM Sales.SalesOrderDetail
GROUP BY ProductID
HAVING SUM(OrderQty) > 1000;
```

Analisa: Fokus pada produk *best seller*.

Latihan 4.4: Filter Rata-Rata Cari ID Penawaran Spesial (SpecialOfferID) dari Sales.SalesOrderDetail yang rata-rata penjualan unitnya (OrderQty) kurang dari 2.

SQL

```
SELECT SpecialOfferID, AVG(OrderQty) AS RataRataBeli
FROM Sales.SalesOrderDetail
GROUP BY SpecialOfferID
HAVING AVG(OrderQty) < 2;
```

Analisa: Mencari promo yang kurang efektif (pembelian rata-rata kecil).

Latihan 4.5: Perbedaan Logika Jelaskan kenapa query ini valid:

SQL

```
SELECT Color
FROM Production.Product
GROUP BY Color
HAVING MAX(ListPrice) > 3000;
```

Analisa: Query ini menampilkan warna yang memiliki setidaknya satu barang mahal (di atas 3000). Meskipun MAX(ListPrice) tidak ditampilkan di SELECT, ia bisa digunakan di HAVING untuk memfilter grup.

BAGIAN 5: MEMBENTUK HASIL (ADVANCED SELECT & ORDER BY) (Step 5 & 6)

Bagian ini menangani DISTINCT (5-2), TOP (5-3), dan Pengurutan (6).

Latihan dan Analisa

Latihan 5.1: Menghilangkan Duplikasi (DISTINCT) Tampilkan daftar semua job title unik dari tabel HumanResources.Employee (atau kolom Color dari Product).

SQL

```
SELECT DISTINCT JobTitle
FROM HumanResources.Employee;
```

Analisa: DISTINCT (Step 5-2) bekerja setelah semua filter selesai. Ia menghapus baris duplikat dari hasil akhir set.

Latihan 5.2: Mengurutkan Data (ORDER BY) Tampilkan produk diurutkan dari harga termahal ke termurah.

SQL

```
SELECT Name, ListPrice
FROM Production.Product
ORDER BY ListPrice DESC;
```

Analisa: ORDER BY (Step 6) adalah langkah terakhir. DESC artinya Descending (Besar ke Kecil), ASC (default) adalah Kecil ke Besar.

Latihan 5.3: Mengambil Data Teratas (TOP) Tampilkan 5 produk termahal.

SQL

```
SELECT TOP 5 Name, ListPrice
FROM Production.Product
ORDER BY ListPrice DESC;
```

Analisa: TOP (Step 5-3) memotong hasil. **Penting:** Selalu gunakan TOP bersamaan dengan ORDER BY. Jika tanpa ORDER BY, data yang diambil "teratas" adalah acak/tidak terprediksi.

Latihan 5.4: Pagination (OFFSET FETCH) Tampilkan produk termahal urutan ke-11 sampai ke-15 (Halaman 2 jika per halaman 10 data).

SQL

```
SELECT Name, ListPrice
FROM Production.Product
ORDER BY ListPrice DESC
OFFSET 10 ROWS
FETCH NEXT 5 ROWS ONLY;
```

Analisa: Fitur modern SQL Server. OFFSET melewati 10 baris pertama, FETCH NEXT mengambil 5 baris berikutnya. Ini solusi standar untuk *paging* di aplikasi web.

Latihan 5.5: Kompleksitas Penuh (All in One) Tampilkan 3 Warna (Color) yang memiliki total nilai stok (ListPrice * jumlah barang) tertinggi, tapi hanya hitung produk yang harganya > 0.

SQL

```
SELECT TOP 3 Color, SUM(ListPrice) AS TotalNilaiStok
FROM Production.Product
WHERE ListPrice > 0 -- Step 2: Filter sampah
GROUP BY Color -- Step 3: Kelompokkan
ORDER BY TotalNilaiStok DESC; -- Step 6: Urutkan
```

Analisa:

1. Ambil data produk yang punya harga.
2. Hitung total harga per warna.
3. Urutkan dari nilai stok terbesar.
4. Ambil 3 teratas. Catatan: TOP dieksekusi di Step 5, setelah GROUP BY tapi sebelum data dikirim ke klien.

TUGAS MANDIRI (RETAIL CHALLENGE)

Gunakan tabel Sales.SalesOrderDetail (Tabel detail transaksi).

1. Tampilkan ProductID dan total uang yang didapat (LineTotal) dari produk tersebut.
2. Hanya hitung transaksi yang OrderQty (jumlah beli) >= 2.
3. Kelompokkan berdasarkan ProductID.
4. Filter agar hanya menampilkan produk yang total uangnya (SUM(LineTotal)) di atas \$50,000.
5. Urutkan dari pendapatan tertinggi.
6. Ambil 10 produk teratas saja.

Jelaskan alur logical query processing dari jawaban Anda!